

Exerciții

Partea I

Implementați următoarele funcționalități, pentru o **listă dublu înlănțuită și circulară**:

1. Inserarea unui element la începutul listei. *insertNodeHeadOfList*
2. Afișarea unei liste. *printList*
3. Afișarea listei în ordine inversă. *printListRev*
4. Inserarea unui element înaintea unui nod precizat printr-o cheie "key". *insertNodeBefore*
5. Inserarea unui element după un nod precizat printr-o cheie "key". *insertNodeAfter*
6. Inserarea unui element la sfârșitul listei. *insertNodeTailOfList*
7. Ștergerea unui element de la începutul listei. *removeNodeHeadOfList*
8. Ștergerea unui element de la finalul listei. *removeNodeTailOfList*
9. Ștergerea unui element din orice loc din listă. *removeNodeFromList*

Partea II

1. Să se scrie o funcție care verifică dacă elementele unei liste reprezintă un șir palindromic. Se numește șir palindromic un șir de numere în care elementele egal depărtate de capete sunt egale.
2. [Reverse a doubly linked list](#)

Partea III

Folosind tool-urile pentru analiza leak-urilor de memorie, corectati următoarele programe:

[15_list.c](#)

```
#include <stdio.h>
#include <stdlib.h>

typedef struct node {
    char *element;
    struct node *next;
} listNode;

void printList(listNode *listStart)
{
    printf("[");
    while (listStart != NULL) {
        printf("%s, ", listStart->element);
        listStart = listStart->next;
    }
}
```

```
    printf("]\n");
}

listNode *getNode(listNode *listNode, int poz)
{
    if (listNode == NULL)
        return NULL;
    for (int i = 0; i < poz; i++) {
        if (listNode->next == NULL)
            break;
        listNode = listNode->next;
    }
    return listNode;
}

void insertNodeHeadOfList(listNode **listStart, char *element)
{
    listNode *aux = (listNode *)malloc(sizeof(listNode));
    if (aux == NULL)
        return;
    aux->element = element;
    aux->next = *listStart;
    *listStart = aux;
}

void insertNodeInList(listNode **listStart, char *element, int poz)
{
    if (poz < 0) {
        printf("Te rugam sa inserezi doar pozitii numere naturale pozitive");
        return;
    }
    if (poz == 0) {
        insertNodeHeadOfList(listStart, element);
        return;
    }
    listNode *prevNode = getNode(*listStart, poz - 1);
    if (prevNode->next == NULL)
        return;
    listNode *aux = (listNode *)malloc(sizeof(listNode));
    if (aux == NULL)
        return;
    aux->element = element;
    aux->next = prevNode->next;
    prevNode->next = aux;
}

void removeNodeHeadOfList(listNode **listStart)
{
    listNode *aux = *listStart;
    *listStart = (*listStart)->next;
}
```

```
    free(aux);
}

void removeNodeFromList(listNode **listStart, int poz)
{
    if (poz == 0) {
        removeNodeHeadOfList(listStart);
        return;
    }
    listNode *prevNode = getNode(*listStart, poz - 1);
    if (prevNode == NULL)
        return;
    listNode *aux = prevNode->next;
    prevNode->next = prevNode->next->next;
    free(aux);
}

int main()
{
    listNode *list = NULL;
    insertNodeHeadOfList(&list, (char *) "Ana");
    printList(list);
    printList(list);
    insertNodeHeadOfList(&list, (char *) "Bob");
    printList(list);
    insertNodeHeadOfList(&list, (char *) "Cris");
    printList(list);
    insertNodeInList(&list, (char *) "Start", 0);
    printList(list);
    insertNodeInList(&list, (char *) "mid", 15);
    printList(list);
    printList(getNode(list, 3));
    printList(list);
    removeNodeHeadOfList(&list);
    printList(list);
    removeNodeFromList(&list, 2);
    printList(list);
    removeNodeFromList(&list, 10);
    printList(list);
    removeNodeFromList(&list, 2);
    printList(list);
}
```

16_list2.c

```
#include <stdio.h>
#include <stdlib.h>
```

```
typedef struct node {
    char *element;
    struct node *next;
} listNode;

void printList(listNode *listStart)
{
    printf("[");
    while (listStart != NULL) {
        printf("%s, ", listStart->element);
        listStart = listStart->next;
    }
    printf("]\n");
}

listNode *getNode(listNode *listNode, int poz)
{
    if (listNode == NULL)
        return NULL;
    for (int i = 0; i < poz; i++) {
        if (listNode->next == NULL)
            break;
        listNode = listNode->next;
    }
    return listNode;
}

void insertNodeHeadOfList(listNode **listStart, char *element)
{
    listNode *aux = (listNode *)malloc(sizeof(listNode));
    if (aux == NULL)
        return;
    aux->element = element;
    aux->next = *listStart;
    *listStart = aux;
}

void insertNodeInList(listNode **listStart, char *element, int poz)
{
    if (poz < 0) {
        printf("Te rugam sa inserezi doar pozitii numere naturale pozitive");
        return;
    }
    if (poz == 0) {
        insertNodeHeadOfList(listStart, element);
        return;
    }
    listNode *prevNode = getNode(*listStart, poz - 1);
    if (prevNode->next == NULL)
        return;
```

```
listNode *aux = (listNode *)malloc(sizeof(listNode));
if (aux == NULL)
    return;
aux->element = element;
aux->next = prevNode->next;
prevNode->next = aux;
}

void removeNodeHeadOfList(listNode **listStart)
{
    listNode *aux = *listStart;
    *listStart = (*listStart)->next;
    free(aux);
}

void removeNodeFromList(listNode **listStart, int poz)
{
    if (poz == 0) {
        removeNodeHeadOfList(listStart);
        return;
    }
    listNode *prevNode = getNode(*listStart, poz - 1);
    if (prevNode->next == NULL)
        return;
    listNode *aux = prevNode->next;
    prevNode->next = prevNode->next->next;
    free(aux);
}

int main()
{
    listNode *list = NULL;
    insertNodeHeadOfList(&list, (char *) "Ana");
    printList(list);
    printList(list);
    insertNodeHeadOfList(&list, (char *) "Bob");
    printList(list);
    insertNodeHeadOfList(&list, (char *) "Cris");
    printList(list);
    insertNodeInList(&list, (char *) "Start", 0);
    printList(list);
    insertNodeInList(&list, (char *) "mid", 15);
    printList(list);
    printList(getNode(list, 3));
    printList(list);
    removeNodeHeadOfList(&list);
    printList(list);
    removeNodeFromList(&list, 2);
    printList(list);
    removeNodeFromList(&list, 10);
    printList(list);
}
```

```
removeNodeFromList(&list, 2);  
printList(list);  
}
```

Upload

TBD

From:

<https://wiki.mta.ro/> - **Cursuri Academia Tehnică Militară "Ferdinand I"**

Permanent link:

https://wiki.mta.ro/c/1/sda/lab/06_new

Last update: **2025/02/27 06:40**